

Problem Set 4 Lab Report: Querying Strings

Overview

Problem Set 4 focused on querying strings in Python by identifying exact and fuzzy substring matches. The assignment required four separate programs that built on one another. The main programming concepts demonstrated were string searching, iteration, recursion, tuples, function design, and testing. Each program used the Python `find()` method to locate substrings inside a larger search string.

The assignment was useful because string processing is common in data science and programming. Textual data may appear in customer reviews, search queries, documents, logs, or business records. Being able to search, count, and compare strings is an important foundation for more advanced text analysis.

Problem One: Counting Substring Matches

The first program created two functions: `countSubstrMatches()` and `countSubstrRecursive()`. Both functions counted how many times a substring appeared inside a larger string. The first function used iteration, while the second function used recursion.

The iterative function used a while loop and repeatedly called `find()` until no additional match was found. Each time a match was located, the count increased by one and the next search began one position after the previous match. This allowed overlapping matches to be counted.

The recursive function solved the same problem differently. Instead of using a loop, the function called itself after each match was found. Each recursive call searched the remaining portion of the string by moving the starting index forward. The base case occurred when `find()` returned `-1`, meaning no match remained.

This problem demonstrated the difference between iterative and recursive solutions. Both approaches produced the same type of result, but the iterative solution was easier to trace, while the recursive solution showed how a larger problem can be reduced into smaller repeated problems.

```
P5 C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\alanp\AppData\Local\Microsoft\WindowsApps\python3.13.exe "//TRUENAS/ShadowRealm/Alan's Prometheus Backup/Education/US College/University of the Cumberland/MSDS-532/assignment_4_1.py"
2
2
0
0
```

The fourth program created the function `fuzzyMatchesOnly()`. This function returned only the starting positions of fuzzy matches where exactly one character was incorrect. Exact matches were excluded from the returned tuple.

The program reused earlier logic from Problems Two and Three. It divided the substring into two parts around each possible wildcard position. It then found where each part appeared in the search string and used `fuzzyMatching()` to identify valid fuzzy matches. After possible matches were found, the program removed exact matches so only fuzzy matches remained.

This problem was the most complex part of the assignment because it required combining multiple functions, handling edge cases, and avoiding duplicate results. It also demonstrated decomposition because a larger problem was solved by breaking it into smaller helper functions.

```
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\alanp\AppData\Local\Microsoft\WindowsApps\python3.13.exe "//TRUENAS/ShadowRealm/Alan's Prometheus Backup/Education/US College/University of the Cumberland/MSDS-532/assignment4_4.py"
(7,)
(5,)
(7, 6)
(8,)
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> |
```

Testing

Testing was included in each script as commented-out code. The testing examples checked exact matches, missing matches, overlapping matches, recursive behavior, and fuzzy matching behavior. The testing code was not deleted because the assignment instructions required testing code to remain in the files while being commented out before submission.

Testing was important because string matching can produce unexpected results when substrings overlap, when case differs, or when no match exists. For example, searching for "Python" inside "python" returns no match because string matching is case-sensitive.

Conclusion

Problem Set 4 demonstrated important Python string-processing techniques. The assignment showed how to count substring matches, collect matching indices, apply recursion, and identify fuzzy matches. The programs also reinforced the importance of modular function design because later problems reused logic from earlier problems.

Overall, the assignment showed that string searching is more than simply finding text. It requires careful attention to indexing, overlapping matches, recursion, function reuse, and testing. These skills are useful for larger programming and data science tasks involving textual data.